COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	09/10/2026	**DATE SUBMITTED:**	09/10/2026
NAME:	Gabriel Immanuel Serrano Vince Andrew Urbano Ellaiza Vizcayno Paterno Miguel Santos Jared Isaiah Dela Cruz	**INSTRUCTOR:**	Engr. Jimlord Quejado

Seatwork 8.1 **Using Sorting Algorithms 2**

1. PROCEDURE OUTPUT

Question	1. Explain the quick sort, shell sort and merge sort types of sorting algorithms.
Answer	**QUICK SORT** - This sorting algorithm picks one item, shuffles everything smaller than it to the left and everything larger to the right, and then repeats that function on the smaller sub-sections. Quick sort is a very fast way of sorting most real-world data, making it a simple, but reliable choice for programmers. **SHELL SORT** - This sorting algorithm is basically an overall better version of insertion sort that fixes its main weakness. Instead of only comparing items right next to each other, it starts by comparing and sorting items that are far apart from each other across the list, and then gradually shrinks that gap down until it is just one. This function lets elements jump quickly to where they roughly belong early on, making it way faster than standard insertion sort for medium-sized lists. **MERGE SORT** - Merge sort follows a system called divide and conquer. It starts off by dividing the list or array into two halves. It halves the list or array until it can no longer be divided. Then the pieces are then merged together, two at a time. During each merge, it compares the front of the right and left pieces and puts the smallest value into the result first. This will continue until the list or array is sorted as one piece again.

Question	2. Give simple sample programs in C++ that uses the above sorting algorithms. Use a user input of 10 integer values in your example elements in an array to be sorted. Explain how the programs work.

Answer	**QUICK SORT:**
	#include <iostream>
	using namespace std;
	
	void swap(int *a, int *b) {
	  int t = *a;
	  *a = *b;
	  *b = t;
	}
	
	void printArray(int array[], int size) {
	  for (int i = 0; i < size; i++)
	  cout << array[i] << " ";
	  cout << endl;
	}
	
	int partition(int array[], int low, int high) {
	  int pivot = array[high];
	  int i = (low - 1);
	
	  for (int j = low; j < high; j++) {
	  if (array[j] <= pivot) {
	  i++;
	  swap(&array[i], &array[j]);
	  }
	  }
	  swap(&array[i + 1], &array[high]);
	  return (i + 1);
	}
	
	void quickSort(int array[], int low, int high) {
	  if (low < high) {
	  int pi = partition(array, low, high);
	  quickSort(array, low, pi - 1);
	  quickSort(array, pi + 1, high);
	  }
	}
	
	int main() {
	  const int SIZE = 10;
	  int data[SIZE];
	
	  cout << "Enter 10 integer values:" << endl;

```
    for (int i = 0; i < SIZE; i++) {
        cout << "Element " << (i + 1) << ": ";
        cin >> data[i];
    }

    cout << "\nUnsorted Array: ";
    printArray(data, SIZE);


    quickSort(data, 0, SIZE - 1);

    cout << "Sorted Array in ascending order: ";
    printArray(data, SIZE);

    return 0;
```

Output:

```
Enter 10 integer values:
Element 1:5

Element 2:4

Element 3:3

Element 4:8

Element 5:1

Element 6:2

Element 7:6

Element 8:7

Element 9:10

Element 10:9
```

```
Unsorted Array: 5 4 3 8 1 2 6 7 10 9
Sorted Array in ascending order: 1 2 3 4 5 6 7 8 9 10
```

	EXPLANATION: The program will first create an array of values. The function named partition is used for selecting the rightmost value as the pivot and it is also responsible for swapping the elements. If the value being compared to the pivot is smaller, it will be moved to the left side of the pivot. Lastly, the quickSort function is responsible for breaking the arrays down to a smaller array.

SHELL SORT:

```cpp
//Shell Sort
#include <iostream>
#include <vector>
using namespace std;

void shellSort(vector<int>& arr) {
    int n = arr.size();

    // Start with a large gap, then reduce it step by step
    for (int gap = n / 2; gap > 0; gap /= 2) {

        // Perform a "gapped" insertion sort for this gap size
        for (int i = gap; i < n; i++) {

            // Current element to be placed correctly
            int temp = arr[i];
            int j = i;

            // Shift elements that are greater than temp to make space
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }

            // Place temp in its correct location
            arr[j] = temp;
        }
    }
}
```

```cpp

// Utility function to print the vector
void printArray(const vector<int>& arr) {
    for (int num : arr)
        cout << num << " ";
    cout << endl;
}

int main() {
    vector<int> arr = {12, 34, 54, 2, 3, 6, 9, 1, 0, 5};

    shellSort(arr);
    printArray(arr);

    return 0;
}
```

```
0 1 2 3 5 6 9 12 34 54
```

EXPLANATION:

In Shell Sort, the program has 10 integer values and stores them in a vector array. The program uses a prototype function called shellSort() to perform the sorting, it works by

starting with the gap of half the array size and reduces the gap while comparing and moving the elements that are far apart. When the gap reaches 1, it works like an insertion sort and arranges all the elements in the correct order. Then , it uses the prototype function called printArray() to display all the sorted values.

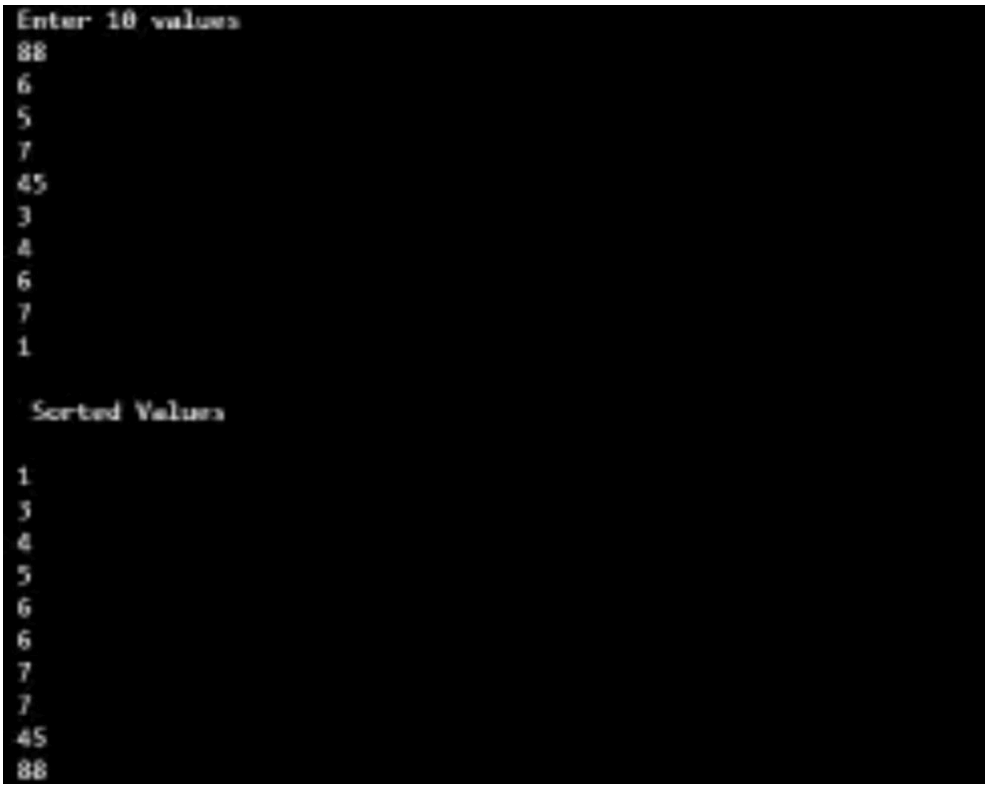
MERGE SORT:

8. Write a program to implement merge sort.

```
//C++ program for merge sort
#include <iostream.h>
// function to merge the two half into a sorted data.
void merge_array(int a[], int low, int high, int mid)
{
   // low to mid and mid+1 to high array are already
sorted
```

```
  int i, j, k;
  int temp_arr[high-low+1];
  i = low;
  k = 0;
  j = mid + 1;
  while (i <= mid && j <= high) // merging of two parts
ıto temp array
  {
     if (a[i] < a[j])
     {
        temp_arr[k] = a[i];
        k++;
        i++;
     }
     else
     {
        temp_arr[k] = a[j];
        k++;
        j++;
     }
  }
  while (i <= mid)    // insertion of remaining values
:om i to mid into temp array.
  {
     temp_arr[k] = a[i];
     k++;
     i++;
  }
  while (j <= high) // insertion of remaining values
:om j to high into temp array.
  {
     temp_arr[k] = a[j];
     k++;
     j++;
  }
  // assign sorted data stored in temp array to an array
  for (i = low; i <= high; i++)
  {
     a[i] = temp_arr[i-low];
  }
```

```
// A function to split array into two parts.
void merge_sort(int a[], int low, int high)
{
    int mid;
    if (low < high)
    {
       mid=(low+high)/2;
       // split array into two parts
       merge_sort(a, low, mid);
       merge_sort(a, mid+1, high);
       // merge arraythem to get sorted values
       merge_array(a, low, high, mid);
    }
}
 void main()
{
int arr[10],i,k;
cout<<"Enter 10 values\n";
for(i=0;i<10;i++)
cin>>arr[i];
//call of merge sort function
    merge_sort(arr, 0, 9);
cout<<" \n Sorted Values  \n";
for(i=0;i<10;i++)
cout<<endl<<arr[i];
}
```



EXPLANATION:
This program sorts 10 numbers using merge sort with two functions. The merge_sort function splits the array into halves by finding the middle index, then calls itself for each half until each half only has one element. Once they can no longer be split, it calls merge_array to combine them in order. The merge_array function compares the values from both halves one by one and puts the smaller value into a temporary array. Once one half runs out, the remaining values from the other half are added since they are already sorted. The temporary array is then copied back into the original array. The main function takes 10 input values, calls merge_sort on the whole array, and then prints the sorted result.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

References:

Merge Sort: https://www.scribd.com/document/574887381/315-14-Lab-Data-Structure-Using-C#page=103
Shell Sort : https://www.geeksforgeeks.org/dsa/shell-sort/

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.